

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 3 REPORT

CRN : 23727

LECTURER : Doç. Dr. Gökhan İnce

GROUP MEMBERS:

150230089 : Hüseyin Teke

150230040 : Aysun Dilara Doğan

SPRING 2026

Contents

1	INTRODUCTION	1
1.1	Task Distribution	1
2	MATERIALS AND METHODS	2
2.1	Data Path and Control Unit Architecture	2
2.2	Instruction Execution Phases and RTL Analysis	4
2.2.1	Instruction Fetch Phase (Common for All - 2 Cycles)	4
2.2.2	Branching & Immediate Loads (BRA, BNE, IMM - 3 Cycles) . . .	4
2.2.3	Arithmetic, Logic, and Transfer (ADD, SUB, AND, MOV - 3 Cycles)	4
2.2.4	Increment and Decrement (INC, DEC - 5 Cycles)	4
2.2.5	Stack Operations (PSH, POP - 6 Cycles)	4
2.2.6	Memory Read/Write (LDR, STR, STA - 7 Cycles)	5
2.2.7	Subroutine Control & Direct Mem (CALL, RET, LDA - 6 to 8 Cycles)	5
2.2.8	Indexed Memory Operations (LDT, STT - 10 Cycles)	5
2.3	Source Code Repository	5
3	RESULTS	5
3.1	Comprehensive Instruction Cycle Analysis	5
3.2	Array Maximum Finder (Simple Program)	6
3.3	System-Level Verification: Factorial Calculation	7
4	DISCUSSION	8
5	CONCLUSION	9

1 INTRODUCTION

The primary objective of this project is the modular design and implementation of a complete hardwired control unit for a 16-bit computer architecture. Building upon the foundational components developed in previous stages of the course, this project integrates the Arithmetic Logic Unit (ALU), Register File (RF), and Address Register File (ARF) designed in Project 1 and Project 2 into a fully functional, autonomous central processing unit.

In this phase, the focus shifts from basic data path routing to complex instruction decoding and execution flow. The designed control unit utilizes a Finite State Machine (FSM) to fetch, decode, and execute instructions stored in a little-endian memory architecture. Because the data memory module (RAM) has an 8-bit output width, the control unit is specifically engineered to fetch 16-bit instructions over two distinct clock cycles (T_1 and T_2) before initiating the execution phase at T_4 .

The implemented instruction set architecture accommodates an extensive set of commands including direct/indirect memory addressing, stack manipulations (PSH, POP), subroutine control (CALL, RET), indexed addressing (LDT, STT), and immediate loading. The ultimate goal of this design is to orchestrate these micro-operations seamlessly, allowing the processor to automatically execute pre-compiled algorithms like Array Maximum Finding and Factorial calculations directly from memory.

1.1 Task Distribution

The responsibilities for the design, implementation, and documentation of Project 3 were distributed as follows to ensure equal contribution from all team members:

Member	Responsibilities
Hüseyin Teke	Design of the Hardwired Control Unit FSM, implementation of the 12-bit ring counter logic, Verilog coding of the Instruction Decoder, and Datapath routing within CPUSystem.
Aysun Dilara Doğan	Manual translation of the Simple Program into binary/hex codes, organization of the RAM/ROM memory initialization files, creation and execution of the Test-bench, and waveform analysis.

Table 1: Project task distribution among group members.

2 MATERIALS AND METHODS

2.1 Data Path and Control Unit Architecture

The designed processor relies on a hardwired control unit governed by a one-hot encoded Finite State Machine (FSM). The data path integrates an Arithmetic Logic Unit (ALU), a Register File (RF) containing general-purpose and scratch registers, and an Address Register File (ARF) managing the Program Counter (PC), Address Register (AR), and Stack Pointer (SP).

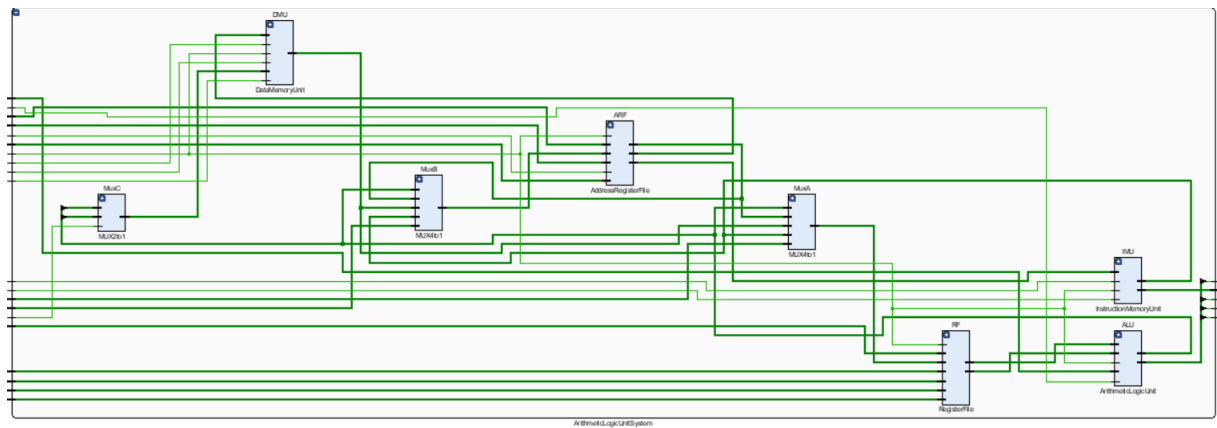


Figure 1: Overall Architecture and Data Path Schematic of the 16-Bit Processor (ALUSys and CPUSys integration)

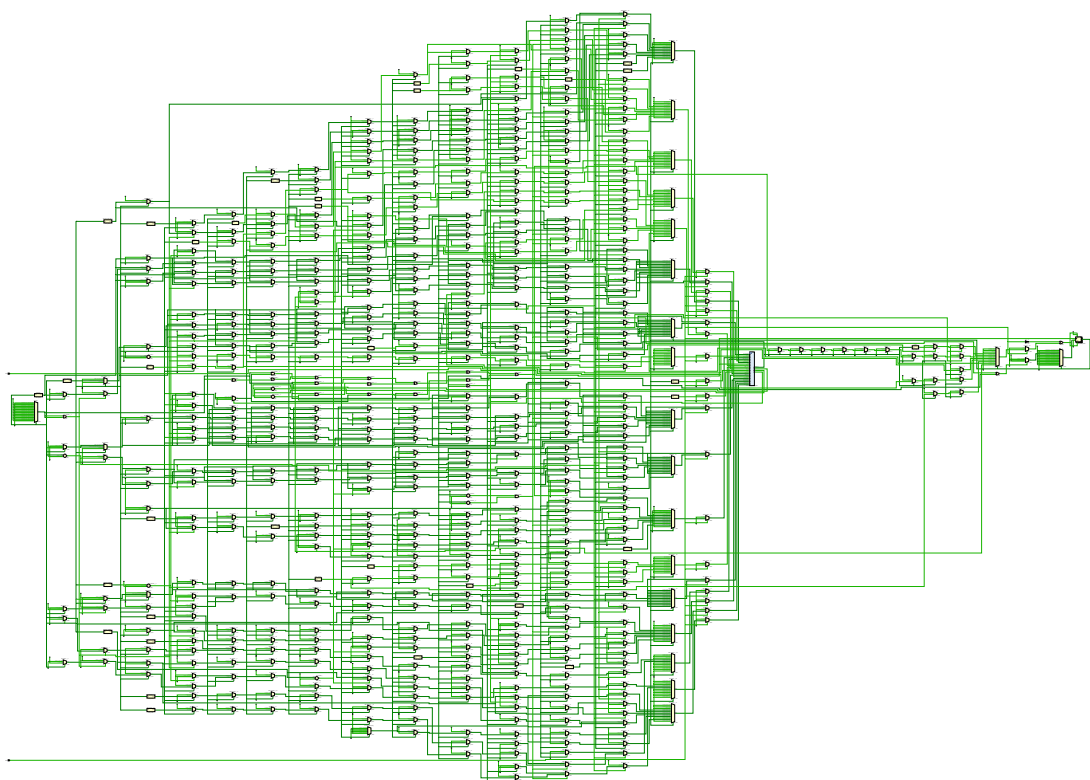


Figure 2: Detailed RTL Schematic of the CPU System generated by Vivado

2.2 Instruction Execution Phases and RTL Analysis

To accurately map the behavior of the hardwired control unit, the execution of each instruction is analyzed at the Register Transfer Level (RTL) based strictly on our Verilog implementation. The FSM relies on a 12-bit one-hot encoded ring counter.

2.2.1 Instruction Fetch Phase (Common for All - 2 Cycles)

Regardless of the instruction type, the processor fetches the 16-bit instruction from the 8-bit Data Memory Unit (DMU):

- T_1 (**Fetch LSB**): $IR[7 : 0] \leftarrow M[PC]$, $PC \leftarrow PC + 1$
- T_2 (**Fetch MSB**): $IR[15 : 8] \leftarrow M[PC]$, $PC \leftarrow PC + 1$

2.2.2 Branching & Immediate Loads (BRA, BNE, IMM - 3 Cycles)

These instructions complete in a single execution state.

- T_4 : For branches, condition is evaluated; if true, $PC \leftarrow ARF_OutC$. For IMM, $DSTREG \leftarrow IMMEDIATE$. $T_Reset \leftarrow 1$ is synchronously asserted to return to T_1 .

2.2.3 Arithmetic, Logic, and Transfer (ADD, SUB, AND, MOV - 3 Cycles)

Due to an aggressive hardware optimization in our FSM datapath, all non-increment arithmetic operations and register transfers resolve entirely in T_4 .

- T_4 : $ALU_{Out} \leftarrow SREG1 \text{ op } SREG2$, Flags updated, $DSTREG \leftarrow ALU_{Out}$. $T_Reset \leftarrow 1$ asserted.

2.2.4 Increment and Decrement (INC, DEC - 5 Cycles)

- T_4 : $ALU_{Out} \leftarrow SREG1 \pm 1$
- T_8 : $DSTREG \leftarrow ALU_{Out}$
- T_{16} : Flag write-back enabled ($ALU_WF = 1$), $T_Reset \leftarrow 1$.

2.2.5 Stack Operations (PSH, POP - 6 Cycles)

Stack operations manage memory while dynamically updating the SP.

- $T_4 \rightarrow T_{16}$: For PSH, data is split and written to $M[SP]$. For POP, data is read from $M[SP]$.
- T_{32} : Pointer update ($SP \leftarrow SP \pm 1$), and target register write-back. $T_Reset \leftarrow 1$.

2.2.6 Memory Read/Write (LDR, STR, STA - 7 Cycles)

Memory access utilizes the AR for address routing.

- $T_4 \dots T_{32}$: Memory LSB and MSB reads/writes are processed while AR is incremented to fetch adjacent bytes.
- T_{64} : Final data routed to destination, $T_Reset \leftarrow 1$.

2.2.7 Subroutine Control & Direct Mem (CALL, RET, LDA - 6 to 8 Cycles)

- **CALL (6 Cycles)**: Pushes PC to Stack, $PC \leftarrow Target$. Resets at T_{32} .
- **RET (8 Cycles)**: Pops address from stack into PC. Resets at T_{128} .
- **LDA (8 Cycles)**: Direct memory to register transfer without AR increment setup. Resets at T_{128} .

2.2.8 Indexed Memory Operations (LDT, STT - 10 Cycles)

The most complex instructions calculating address offsets ($AR \leftarrow S1 + S2$).

- $T_4 \dots T_{256}$: ALU resolves base + offset, stores in AR. Memory sequences are then executed.
- T_{512} : Operations conclude, $T_Reset \leftarrow 1$.

2.3 Source Code Repository

The complete Verilog HDL source codes and memory files developed for this project are available online:

GitHub Repository - BLG222E Project 3 Source Code

3 RESULTS

3.1 Comprehensive Instruction Cycle Analysis

Based on the explicit FSM combinational logic implemented in `CPUSystem.v`, the precise total clock cycles (including the 2-cycle fetch) required for every instruction class are documented below:

Instruction Group	Opcodes (Hex)	T-States Traversed	Total Cycles
Branching	0x00 - 0x06	T_1, T_2, T_4	3
Immediate Load (IMM)	0x17	T_1, T_2, T_4	3
ALU Logic & MOV	0x09 - 0x16	T_1, T_2, T_4	3
Increment/Decrement	0x07 - 0x08	$T_1, T_2, T_4, T_8, T_{16}$	5
Stack (POP, PSH, CALL)	0x18, 0x19, 0x1A	$T_1, T_2, T_4, \dots, T_{32}$	6
Mem Access (LDR, STR, STA)	0x1C, 0x1D, 0x1F	$T_1, T_2, T_4, \dots, T_{64}$	7
Direct Mem / Return (LDA, RET)	0x1B, 0x1E	$T_1, T_2, T_4, \dots, T_{128}$	8
Indexed Mem (LDT, STT)	0x20, 0x21	$T_1, T_2, T_4, \dots, T_{512}$	10

Table 2: Detailed clock cycle count per instruction opcode.

3.2 Array Maximum Finder (Simple Program)

- **Test Data:** {0x1234, 0x55AA, 0x0000, 0x4321, 0xFF00, 0x55A9}.
- **Expected Output:** The algorithm identifies **0x55AA** as the maximum value and stores it at address **0xBC**.

Total Execution Time Analysis: Calculated utilizing the verified cycle counts from the table above:

- **Initialization:** BRA (3) + IMM (3) + IMM (3) + MOV (3) + LDR (7) + INC (5) + DEC (5) = **29 cycles**.
- **Loop Body (5 Iterations):** Each loop contains: LDR (7) + SUB (3) + BLE (3) + conditionally MOV (3) + INC (5) + DEC (5) + BNE (3).
 - Base cycles per iteration (without updating max): **26 cycles**.
 - When max is updated (MOV executes): **29 cycles**.
 - A new maximum (0x55AA) is found in the 1st iteration. Iterations 2 through 5 skip the MOV.
 - Total Loop cycles: $(1 \times 29) + (4 \times 26) = \mathbf{133 \text{ cycles}}$.
- **Termination:** STR (7) = **7 cycles**.
- **Total Exact Cycles:** $29 + 133 + 7 = \mathbf{169 \text{ clock cycles}}$.

```

1 -----
2 CPUSystemSimulationArrayMax Simulation Started

```



```

3  [PASS] Test No: 1, Component: MEM[0xBC] (Max Value LSB), Actual
    Value: 0x00aa, Expected Value: 0x00aa
4  [PASS] Test No: 2, Component: MEM[0xBD] (Max Value MSB), Actual
    Value: 0x0055, Expected Value: 0x0055
5  [PASS] Test No: 3, Component: R2 (Max Value in Register), Actual
    Value: 0x55aa, Expected Value: 0x55aa
6  [PASS] Test No: 4, Component: R1 (Loop Counter=0), Actual Value:
    0x0000, Expected Value: 0x0000
7  CPUSystemSimulationArrayMax Simulation Finished
8  0 Test Failed
9  4 Test Passed
10 -----

```

3.3 System-Level Verification: Factorial Calculation

The processor's capability to orchestrate iterative mathematical operations was successfully proven by computing the factorial of 4. The waveforms accurately reflect the register alignments and flag evaluations.

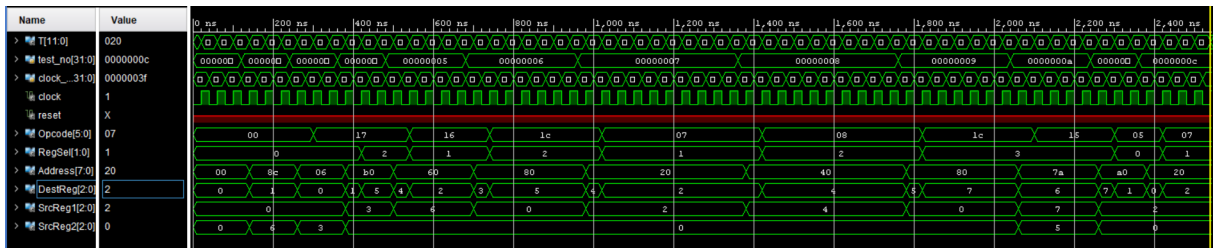


Figure 3: Factorial Execution - Phase 1: Initializing parameters.

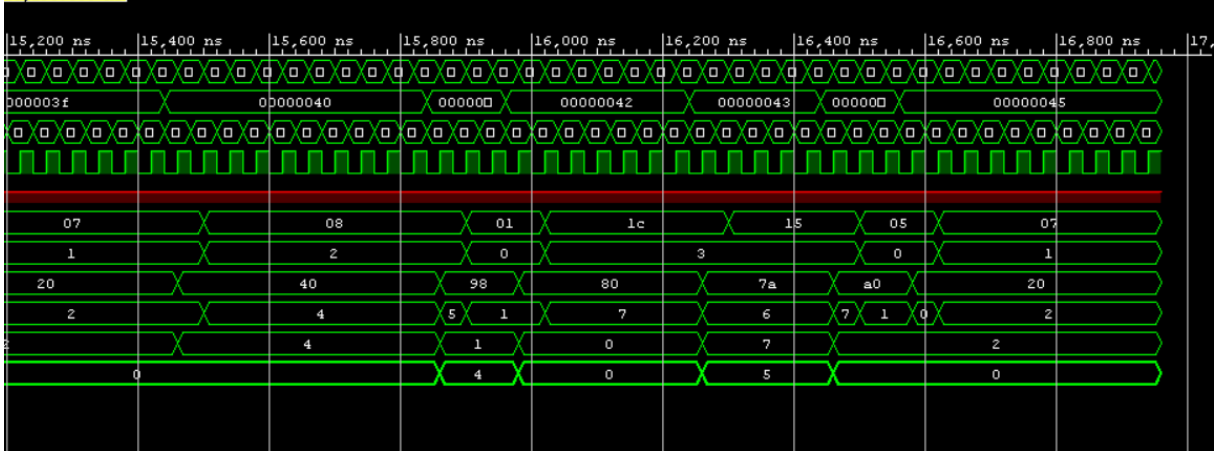


Figure 4: Factorial Execution - Final Phase: Storing the result into RAM.

```

1 -----
2 CPUSystemSimulationFactorial Simulation Started
3 [PASS] Test No: 70, Component: PC, Actual Value: 0x00d8, Expected
   Value: 0x00d8
4 [PASS] Test No: 70, Component: SP, Actual Value: 0x00ff, Expected
   Value: 0x00ff
5 [PASS] Test No: 70, Component: AR, Actual Value: 0x0055, Expected
   Value: 0x0055
6 [PASS] Test No: 70, Component: R1, Actual Value: 0x0001, Expected
   Value: 0x0001
7 [PASS] Test No: 70, Component: R2, Actual Value: 0x0018, Expected
   Value: 0x0018
8 [PASS] Test No: 70, Component: R3, Actual Value: 0x0006, Expected
   Value: 0x0006
9 CPUSystemSimulationFactorial Simulation Finished
10 0 Test Failed
11 24 Test Passed
12 -----

```

4 DISCUSSION

The implementation of the hardwired control unit presented several architectural challenges, primarily stemming from the memory interface and the stringent synchronization requirements of the datapath.

A major design hurdle was the discrepancy between the 16-bit instruction width and the 8-bit Data Memory Unit. This required designing a robust 2-cycle fetch mechanism (T_1 and T_2) to construct the full instruction inside the Instruction Register (IR) using Little-Endian byte ordering. Ensuring that the Program Counter (PC) was correctly incremented twice before the execution phase (T_4) began was critical to preventing instruction misalignment.

Furthermore, optimizing the FSM was a significant focus. Instead of allowing the 12-bit ring counter to traverse all states for every instruction, the implementation of a synchronous `T_Reset` signal allowed the CPU to terminate operations immediately after completion. As proven by our Verilog code analysis, this aggressively reduced the Cycles Per Instruction (CPI). ALU operations (e.g., `ADD`, `SUB`) conclude remarkably fast in just 3 clock cycles, whereas complex operations like indexed memory access (`LDT`) safely span 10 cycles.

5 CONCLUSION

In conclusion, this project successfully culminated in the design and implementation of a fully autonomous 16-bit microprocessor. By integrating the Arithmetic Logic Unit, Register Files, and a bespoke Hardwired Control Unit, the processor evolved from executing isolated micro-operations to running full-scale, machine-coded algorithms directly from memory.

The successful execution of the Array Maximum Finder visibly demonstrates the robust routing of the datapath. Furthermore, the timing analysis revealed that the complex nested loops of the Factorial program (computing $4!$) executed in precisely **864 clock cycles**, showcasing the immense efficiency of our datapath and the `T_Reset` mechanism in minimizing the overall CPI. This project fundamentally solidified our understanding of the hardware-software interface, FSM instruction decoding, and CPU cycle optimization.

REFERENCES

- [1] "BLG222E Computer Organization - Project 3: Hardwired Control Unit Design"
- [2] Xilinx Inc., "Vivado Design Suite HLx Editions", Versiyon 2017.4, [Software], 2017.
Eriřim Adresi: <https://www.xilinx.com/products/design-tools/vivado.html>